ETSIIT
Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación

Ingeniería Informática + ADE

Universidad de Granada (UGR)

Autor: Ismael Sallami Moreno

Asignatura: Ejercicios Tema 3 SCD: Paso de mensajes

Índice

1. Ejercicio 77 **3**
1.1. Enunciado 3
1.2. Solucion 3

2. Ejercicio 78 **5**
2.1. Enunciado 5
2.2. Solución 6

3. Ejercicio 79 **7**
3.1. Enunciado 7
3.2. Solución 7

1 Ejercicio 77

1.1. Enunciado

En un sistema distribuido, 6 procesos clientes necesitan sincronizarse de forma específica para realizar cierta tarea, de forma que dicha tarea sólo podrá ser realizada cuando tres procesos estén preparados para realizarla. Para ello, envían peticiones a un proceso controlador del recurso y esperan respuesta para poder realizar la tarea específica. El proceso controlador se encarga de asegurar la sincronización adecuada. Para ello, recibe y cuenta las peticiones que le llegan de los procesos, las dos primeras no son respondidas y producen la suspensión del proceso que envía la petición (debido a que se bloquea esperando respuesta) pero la tercera petición produce el desbloqueo de los tres procesos pendientes de respuesta. A continuación, una vez desbloqueados los tres procesos que han pedido (al recibir respuesta), inicializa la cuenta y procede cíclicamente de la misma forma sobre otras peticiones. El código de los procesos clientes aparece aquí abajo. Los clientes usan envío asíncrono seguro para realizar su petición, y esperan con una recepción síncrona antes de realizar la tarea:

```
process Cliente[ i : 0..5 ] ;
begin
  while true do begin
    send( peticion, Controlador );
    receive( permiso, Controlador );
    Realiza_tarea_grupal();
  end
end

process Controlador ;
begin
  while true do begin
    ...
  end
end
```

Describir en pseudocódigo el comportamiento del proceso controlador, utilizando una orden de espera selectiva que permita implementar la sincronización requerida entre los procesos. Es posible utilizar una sentencia del tipo select for i=... to ... para especificar diferentes ramas de una sentencia selectiva que comparten el mismo código dependiente del valor de un índice i.

1.2. Solucion

En un sistema distribuido, 6 procesos clientes necesitan sincronizarse de forma específica para realizar cierta tarea, de forma que dicha tarea sólo podrá ser realizada cuando tres procesos estén preparados para realizarla. Para ello, envían peticiones a un proceso controlador del recurso y esperan respuesta para poder realizar la tarea específica. El proceso controlador se encarga de asegurar la sincronización adecuada.

El controlador cuenta las peticiones que le llegan de los procesos:

- Las dos primeras peticiones no son respondidas, lo que provoca que los procesos que las envían queden bloqueados esperando una respuesta.

- La tercera petición produce el desbloqueo de los tres procesos pendientes, enviándoles una respuesta.

Una vez desbloqueados los tres procesos, el controlador inicializa la cuenta y procede cíclicamente de la misma forma para nuevas peticiones. Los clientes usan envío asíncrono seguro para realizar su petición y esperan con una recepción síncrona antes de realizar la tarea.

El código de los procesos clientes es el siguiente:

```
process Cliente[ i : 0..5 ] ;
begin
  while true do begin
    send( peticion, Controlador );   // Envía una petición al proceso controlador.
    receive( permiso, Controlador ); // Espera un permiso del controlador.
    Realiza_tarea_grupal();          // Realiza la tarea grupal una vez recibido el permiso.
  end
end
```

Listing 1: Código de los procesos clientes

El comportamiento del proceso controlador, que asegura la sincronización, se describe en pseudocódigo a continuación:

```
process Controlador ;
var
  contador : integer := 0;           // Cuenta las peticiones recibidas.
  buffer : array[0..2] of integer;   // Almacena los índices de los procesos en espera.

begin
  while true do begin
    select
      for i := 0 to 5                // Itera sobre las peticiones de los procesos clientes.
      when receive( peticion, Cliente[i] ) do
        buffer[contador] := i;       // Almacena el índice del proceso cliente en el buffer.
        contador := contador + 1;    // Incrementa el contador de peticiones.

        if contador = 3 then begin   // Si se han recibido tres peticiones ...
          for j := 0 to 2 do
            send( permiso, Cliente[buffer[j]] ); // Envía permiso a los tres procesos.
          contador := 0;             // Reinicia el contador.
        end
    end
  end
end
```

Listing 2: Pseudocódigo del proceso Controlador

Explicación del código del controlador:

- El proceso controlador mantiene un contador de peticiones y un buffer que almacena los índices de los clientes en espera.
- Por cada petición recibida, almacena el índice del cliente en el buffer y aumenta el contador.
- Cuando el contador alcanza tres, el controlador envía permisos a los tres procesos almacenados en el buffer y reinicia el contador.

Este enfoque asegura que los clientes se sincronizan correctamente antes de realizar la tarea grupal, tal como se requiere en el enunciado.

2 Ejercicio 78

2.1. Enunciado

En un sistema distribuido, 3 procesos productores producen continuamente valores enteros y los envían a un proceso buffer que los almacena temporalmente en un array local de 4 celdas enteras para ir enviándoselos a un proceso consumidor. A su vez, el proceso buffer realiza lo siguiente, sirviendo de forma equitativa al resto de procesos:

- Envía enteros al proceso consumidor siempre que su array local tenga al menos dos elementos disponibles.
- Acepta envíos de los productores mientras el array no esté lleno, pero no acepta que cualquier productor pueda escribir dos veces consecutivas en el búfer.

El código del productor y del consumidor es el siguiente:

```
process Productor[ i : 0..2 ] ;
var dato : integer ;
begin
  while true do begin
    dato := Producir();
    send( dato, Buffer );
  end
end

process Consumidor ;
begin
  while true do begin
    receive ( dato, Buffer );
    Consumir( dato );
  end
end

process Buffer ;
begin
  while true do begin
    ...
  end
end
```

Se pide: describir en pseudocódigo el comportamiento del proceso buffer, utilizando una orden de espera selectiva que permita implementar la sincronización requerida entre los procesos.

2.2. Solución

```
Process Buffer;
var
  buffer : array[0..3] of integer; // Array local de 4 celdas enteras.
  count : integer := 0;             // Contador de elementos en el buffer.
  lastProducer : integer := -1;     // Índice del último productor que escribió en el buffer.

  begin

    while true do begin

      select
        when receive(dato, Productor[0]) do
          if(count <4 and lastProducer != 0) then
            buffer[count] := dato; // Almacena el dato en el buffer.
            count := count + 1;   // Incrementa el contador de elementos.
            lastProducer := 0; // Actualiza el índice del último productor.
          end

        when receive(dato, Productor[1]) do
          if(count <4 and lastProducer != 1) then
            buffer[count] := dato; // Almacena el dato en el buffer.
            count := count + 1;   // Incrementa el contador de elementos.
            lastProducer := 1; // Actualiza el índice del último productor.
          end

        when receive(dato, Productor[2]) do
          if(count <4 and lastProducer != 2) then
            buffer[count] := dato; // Almacena el dato en el buffer.
            count := count + 1;   // Incrementa el contador de elementos.
            lastProducer := 2; // Actualiza el índice del último productor.
          end

        when count >= 2 do // Si hay al menos dos elementos en el buffer...
          send(buffer[0], Consumidor); // Envía el primer elemento al consumidor.
          for i := 0 to 2 do
            buffer[i] := buffer[i + 1]; // Desplaza los elementos restantes.
          end
          count := count - 1; // Decrementa el contador de elementos.


```

```
      end


    end
```

Listing 3: Pseudocódigo del proceso Buffer

3 Ejercicio 79

3.1. Enunciado

Suponer un proceso productor y 3 procesos consumidores que comparten un buffer acotado de tamaño B. Cada elemento depositado por el proceso productor debe ser retirado por todos los 3 procesos consumidores para ser eliminado del buffer. Cada consumidor retirará los datos del buffer en el mismo orden en el que son depositados, aunque los diferentes consumidores pueden ir retirando los elementos a ritmo diferente unos de otros.

Por ejemplo, mientras un consumidor ha retirado los elementos 1, 2 y 3, otro consumidor puede haber retirado solamente el elemento 1. De esta forma, el consumidor más rápido podría retirar hasta B elementos más que el consumidor más lento.

Describir en pseudocódigo el comportamiento de un proceso que implemente el buffer de acuerdo con el esquema de interacción descrito usando una construcción de espera selectiva, así como el del proceso productor y de los procesos consumidores. Comenzar identificando qué información es necesario representar, para después resolver las cuestiones de sincronización.

Una posible implementación del buffer mantendría, para cada proceso consumidor, el puntero de salida y el número de elementos que quedan en el buffer por consumir:

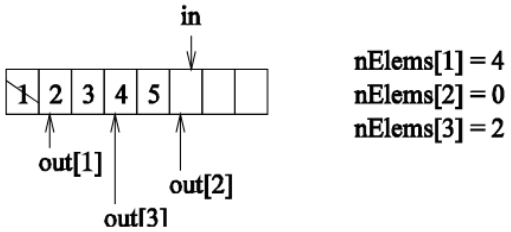

Figura 1: Esquema de interacción entre el productor y los consumidores.

3.2. Solución

```
process Buffer;

var
  out: array[0..2] of integer; // Punteros de salida para cada consumidor
      .
  nElems: array[0..2] of integer; // Número de elementos que quedan por
      consumir para cada consumidor.
```

```
    B: integer; // Tamaño del buffer.
    buf: array[0..B-1]; // Buffer acotado de tamaño B.
    begin
    select for i:=0 to 2 do

      when nElemens[i] != 0 do // Si el consumidor i tiene elementos por consumir...
        send (buf[out[i]], consumidor[i]); // Envía el elemento al consumidor i.
        out[i] + 1 mod B; // Actualiza el puntero de salida del consumidor i.
        nElemens[i] = nElemens[i]-1; // Decrementa el número de elementos por consumir del consumidor i.
      end do

      when nElemens[0] != B or nElemens[1] != B or nElemens[2] != B do // Si el buffer no está lleno para algún consumidor...
        receive(dato, productor); // Recibe un dato del productor.
        bufer[i] = dato; // Almacena el dato en el buffer.
        for j:=0 to 2 do
          nElemens[j] = nElemens[j] + 1; // Incrementa el número de elementos por consumir para cada consumidor.
        end do
      end do
    end select
  end
```

Listing 4: Pseudocódigo del proceso Buffer

Explicación del código del proceso Buffer

El proceso Buffer se encarga de gestionar la interacción entre un productor y tres consumidores que comparten un buffer acotado de tamaño B. Cada elemento depositado por el productor debe ser retirado por todos los consumidores para ser eliminado del buffer. La idea general del código es mantener un buffer acotado de tamaño B donde el productor puede depositar elementos y los consumidores pueden retirarlos. Cada consumidor tiene su propio puntero de salida y contador de elementos por consumir. El proceso Buffer asegura que cada elemento depositado por el productor sea retirado por todos los consumidores antes de ser eliminado del buffer. Además, se asegura de que el buffer no se llene completamente para ningún consumidor y que los consumidores retiren los elementos en el mismo orden en el que fueron depositados.